

ENGENHARIA REVERSA COM APLICATIVOS DE CÓDIGO ABERTO EM AMBIENTES EXPERIMENTAIS CONTROLADOS

Enzo Ferroni (IC) e Rodrigo Cardoso Silva (Orientador)

Apoio: PIBIC Mackenzie

RESUMO

Aplicativos de celular chegam ao usuário como arquivos prontos para executar, quase sempre sem o código que lhes deu origem. Auditar sua segurança exige, então, reconstruir o funcionamento a partir do produto acabado. Este artigo apresenta um estudo experimental sobre essa reconstrução no ecossistema iOS. O objetivo foi avaliar se métodos repetíveis de análise de binários Mach-O ARM64 são viáveis usando apenas ferramentas de código aberto. A pesquisa ocorreu em laboratório controlado, restrita a aplicativos abertos da suíte educacional da OWASP. Essa delimitação preserva a conformidade com a legislação brasileira e com as cláusulas contratuais da plataforma. Na etapa estática, Radare2 e Rabin2 produziram contagens repetíveis de funções, cadeias de caracteres sensíveis e chamadas passíveis de observação em quatro binários. O Ghidra apoiou a reconstrução do fluxo interno das rotinas críticas. Na etapa dinâmica, o Frida foi executado em três lotes de trinta tentativas por técnica, somando noventa por cenário. A instalação do interceptador funcionou em todas as tentativas, nas três técnicas observadas: validação de transporte, Keychain e chamadas de criptografia. Esse resultado mede a viabilidade da observação no Simulador, e não fragilidade dos aplicativos. O dado informativo é a dispersão: o tempo até o interceptador ficar pronto manteve-se abaixo de 0,12 segundo, com coeficiente de variação de 8% em duas técnicas e de 29,8% na terceira, concentrada num único valor extremo da primeira tentativa. Conclui-se que ferramentas abertas bastam para auditoria acadêmica rigorosa, desde que a medida seja lida como observabilidade. O instrumento foi validado por controles negativos.

Palavras-chave: Engenharia reversa. Segurança móvel. iOS.

ABSTRACT

Mobile applications reach users as ready-to-run files, almost always without the source code behind them. Auditing their security therefore requires reconstructing how they work from the finished product. This article presents an experimental study of that reconstruction in the iOS ecosystem. The goal was to assess whether repeatable methods for analysing Mach-O ARM64 binaries are feasible using open-source tools alone. The research took place in a controlled laboratory, restricted to open applications from the OWASP educational suite. This boundary preserves compliance with Brazilian law and with the platform contractual terms. In the static stage, Radare2 and Rabin2 produced repeatable counts of functions, sensitive

strings and observable calls across four binaries. Ghidra supported the reconstruction of the internal flow of critical routines. In the dynamic stage, Frida ran in three batches of thirty attempts per technique, totalling ninety per scenario. Interceptor installation succeeded in every attempt, across the three observed techniques: transport validation, Keychain and cryptographic calls. This result measures the feasibility of observation in the Simulator, not application weakness. The informative figure is dispersion: the time until the interceptor became ready stayed below 0.12 second, with a coefficient of variation of 8% in two techniques and 29.8% in the third, the latter concentrated in a single extreme value from the first attempt. It is concluded that open tools suffice for rigorous academic auditing, provided the measure is read as observability. The instrument was validated by negative controls.

Keywords: Reverse engineering. Mobile security. iOS.

INTRODUÇÃO

Um aplicativo de celular chega ao usuário como um arquivo pronto para executar. O código que lhe deu origem quase nunca acompanha esse arquivo. Quem precisa auditar a segurança do aplicativo não tem, portanto, o que ler: precisa partir do produto acabado e reconstruir como ele funciona por dentro. A essa reconstrução dá-se o nome de engenharia reversa. A prática nasceu na indústria, voltada à compatibilidade entre sistemas, e hoje ocupa lugar central no estudo de programas maliciosos e na auditoria de segurança (EGELE et al., 2012; TAM et al., 2017).

No caso da Apple, dois movimentos recentes tornaram essa reconstrução mais difícil. O primeiro foi a migração de toda a linha de produtos para a arquitetura ARM64. O segundo foi o fechamento progressivo da plataforma, que restringe o que um programa externo pode observar. Some-se a isso uma barreira que não é técnica: o contrato de licença da empresa proíbe desmontar seus aplicativos, mesmo para fins educacionais ou científicos (APPLE INC., 2023). A legislação brasileira admite a prática com finalidade acadêmica (BRASIL, 1998; SCHIRRU, 2014), mas o conflito com as cláusulas contratuais limita o que se pode efetivamente testar. Daí o problema desta pesquisa: é possível auditar aplicativos iOS com rigor metodológico e sem infringir contratos, usando apenas ferramentas de código aberto?

A saída adotada foi escolher com cuidado o objeto de estudo. Aplicativos de código aberto podem ser inspecionados livremente, porque a própria licença autoriza. Eles permitem exercitar as mesmas técnicas sem esbarrar em nenhum contrato. O trabalho justifica-se, ainda, por oferecer um procedimento documentado e repetível, útil a outros pesquisadores e a equipes de resposta a incidentes. Nessa área, um achado só tem valor se outra pessoa conseguir chegar ao mesmo resultado.

O objetivo geral é aplicar técnicas de engenharia reversa a aplicativos de código aberto, avaliando como as ferramentas se comportam em um ambiente moderno e controlado. Cinco objetivos específicos detalham essa meta. O primeiro é selecionar ferramentas abertas adequadas a binários ARM64. O segundo, montar o laboratório onde os ensaios ocorrem. O terceiro, aplicar as duas formas de análise sobre os binários escolhidos. O quarto, medir o desempenho da observação por meio de repetições controladas. O quinto, reunir o aprendizado em um conjunto de boas práticas.

REFERENCIAL TEÓRICO

Engenharia reversa de software

A engenharia reversa de software consiste em examinar um sistema para identificar seus componentes e a relação entre eles, produzindo uma descrição em nível mais alto do que o código executável (SOMMERVILLE, 2011). O caminho é o inverso do desenvolvimento comum: em vez de partir da especificação para o programa, parte-se do programa pronto para entender sua estrutura. Sikorski e Honig (2012) dividem a prática em duas técnicas. Na análise estática, o código é examinado sem ser executado, por meio de desmontagem e leitura do fluxo de dados. Na análise dinâmica, o programa é observado enquanto roda, com apoio de depuração e rastreamento. As duas se completam. A primeira alcança todo o programa, mas não garante que os caminhos encontrados sejam de fato percorridos. A segunda mostra o que realmente acontece, porém só nos trechos exercitados durante o teste.

Análise de segurança em plataformas móveis

Tam et al. (2017) revisam as técnicas de detecção de programas maliciosos em plataformas móveis e mostram como ameaças e defesas evoluem em resposta uma à outra. Android e iOS impõem obstáculos distintos a quem analisa. O primeiro apoia-se em permissões e na separação entre processos (ENCK et al., 2011). O segundo trabalha como um ecossistema fechado, com assinatura obrigatória de código. No iOS, ainda é comum que o próprio aplicativo tente detectar se está sendo examinado. A documentação da fabricante descreve as camadas de defesa que limitam o alcance de qualquer observação feita no aparelho (APPLE INC., 2026).

Ferramentas de análise

Ferramentas abertas tornaram essas técnicas acessíveis fora da indústria. O Ghidra, criado pela Agência de Segurança Nacional dos Estados Unidos e liberado em 2019, equipara-se a alternativas comerciais na leitura de binários (EAGLE; NANCE, 2020; NATIONAL SECURITY AGENCY, 2025). O Frida trouxe um avanço expressivo do outro lado: permite inserir trechos de código em um programa que já está rodando, sem alterar o arquivo original (FRIDA TEAM, 2026). O Radare2 complementa o Ghidra na inspeção de baixo nível, como cabeçalhos e símbolos (RADARE2 PROJECT, 2024), e o Objection facilita a exploração interativa do aplicativo em funcionamento (SENSEPOST, 2026). Como referência de avaliação, a OWASP mantém um guia de testes e um padrão de verificação para segurança móvel (OWASP FOUNDATION, 2024a, 2024b).

Modelos de linguagem aplicados à análise de binários

Trabalhos recentes investigam o uso de modelos de linguagem para acelerar a leitura de código binário, sobretudo quando o programa não traz nomes de funções (CHEN et al., 2026; HU et al., 2026). Os mesmos estudos ponderam que a conferência humana continua necessária. Diante de código deliberadamente embaralhado, o desempenho desses modelos

ainda é limitado (PORDANESH; TAN, 2024; PROMON SECURITY RESEARCH TEAM, 2026). Este artigo trata o tema como contexto, não como método: a análise aqui é conduzida e validada por pessoas.

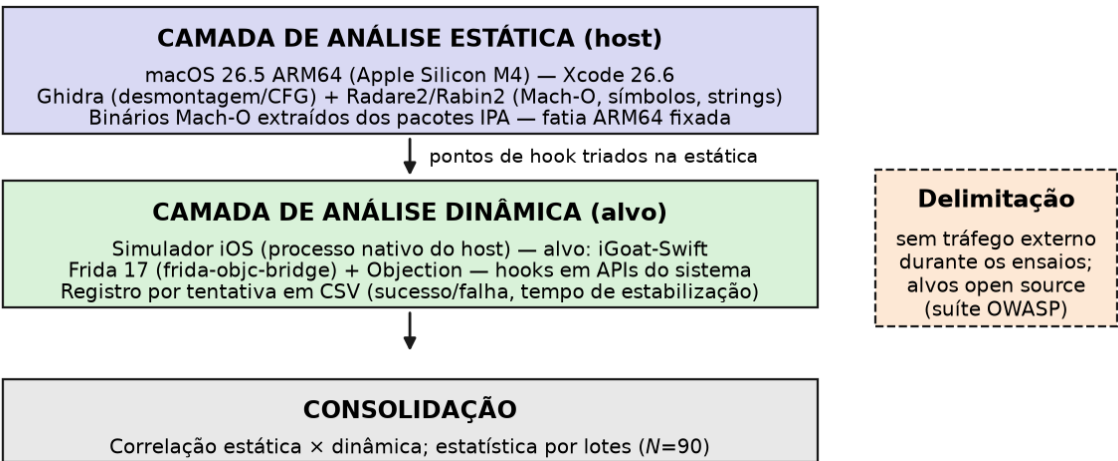
METODOLOGIA

A pesquisa é um estudo experimental de laboratório, com abordagem ao mesmo tempo quantitativa e qualitativa. Quatro decisões orientaram o desenho e todas visam à repetibilidade. A primeira foi restringir os alvos a aplicativos de código aberto. A segunda foi definir duas medidas principais: a taxa de sucesso da observação e o tempo que o programa leva para se estabilizar depois de instrumentado. A terceira foi executar os ensaios em lotes repetidos, o que permite medir a variação entre eles. A quarta foi confrontar de forma explícita o que a leitura do binário sugere com o que a execução confirma. Scripts, planilhas e roteiros estão em repositório público (FERRONI; SILVA, 2026), de modo que qualquer pessoa pode conferir os números apresentados adiante.

Ambiente experimental

O laboratório tem duas camadas, representadas na Figura 1. Na camada estática, uma estação macOS com processador Apple Silicon executa Ghidra e Radare2 sobre os binários extraídos dos pacotes de instalação. Na camada dinâmica, o aplicativo roda no Simulador iOS do Xcode e é observado pelo Frida. Durante os ensaios não houve tráfego para fora da máquina. Os alvos foram apenas aplicativos abertos, condição que define o limite jurídico do experimento (BRASIL, 1998; APPLE INC., 2023).

Figura 1. Arquitetura do ambiente experimental controlado



Fonte: elaborada pelos autores (2026).

Seleção dos alvos

O plano registrado no início da pesquisa previa examinar também binários compilados pelo próprio pesquisador. Ao longo da execução, optou-se por concentrar os ensaios na suíte educacional da OWASP, decisão que é ela própria um resultado e está justificada adiante. Foram analisados quatro binários dessa suíte: DVIA-v2, iGoat-Swift e as duas versões do UnCrackable App, níveis 1 e 2. Três razões sustentam a escolha. A primeira é a repetibilidade: são alvos usados em laboratórios de segurança no mundo todo, o que permite comparar resultados. A segunda é que suas falhas já estão documentadas, o que oferece um parâmetro para conferir os achados (GIANCHANDANI, 2025; OWASP FOUNDATION, 2024c, 2025). A terceira é evitar um viés: um binário recém-compilado pelo próprio pesquisador tende a ser simples demais e a não reproduzir as defesas que aparecem em aplicativos comerciais, como as verificações de ambiente e a resistência à depuração.

Procedimento da análise estática

Na etapa estática, cada binário foi lido como um arquivo Mach-O, sem ser executado, sempre na fatia ARM64. Extraíu-se o executável principal de cada pacote, identificando o tipo de arquivo, a presença de múltiplas arquiteturas e os comandos de carga relevantes. No Ghidra, observou-se que parte dos nomes de classes e métodos sobreviveu à compilação, o que facilita reconhecer a estrutura do programa (EAGLE; NANCE, 2020). Os metadados de Swift exigiram tratamento próprio, por serem mais opacos. Aplicou-se a reversão da decoração de símbolos (demangling), que converte identificadores como `$s11iGoat_Swift20TransitionAnimatableP` no nome legível do protocolo `TransitionAnimatable`. Inspeccionaram-se as seções de reflexão (reflection, em `__swift5_types` e `__swift5_fieldmd`) para recuperar tipos e campos. Leram-se as tabelas de testemunho de protocolo (protocol witness tables), que indicam qual implementação atende a cada protocolo e são indispensáveis para seguir chamadas resolvidas apenas em tempo de execução. Padrões de concorrência estruturada (async/await) apareceram como funções parciais e estruturas de continuação, que quebram o fluxo do programa em pedaços e exigem correlação adicional.

Os indicadores dessa etapa foram extraídos de forma automática com Radare2 e Rabin2, em três medidas comparáveis entre alvos. A primeira é o número de funções recuperadas. A segunda é a quantidade de cadeias de caracteres sensíveis, obtida por uma listagem completa filtrada por termos de segurança. A terceira são os símbolos instrumentáveis, isto é, as chamadas ligadas a Keychain, validação de transporte e criptografia. Por serem contagens automáticas, e não seleções feitas à mão, outro pesquisador que repita o procedimento chega aos mesmos valores. A Tabela 1 reúne os resultados.

Tabela 1. Indicadores consolidados da análise estática por aplicativo-alvo

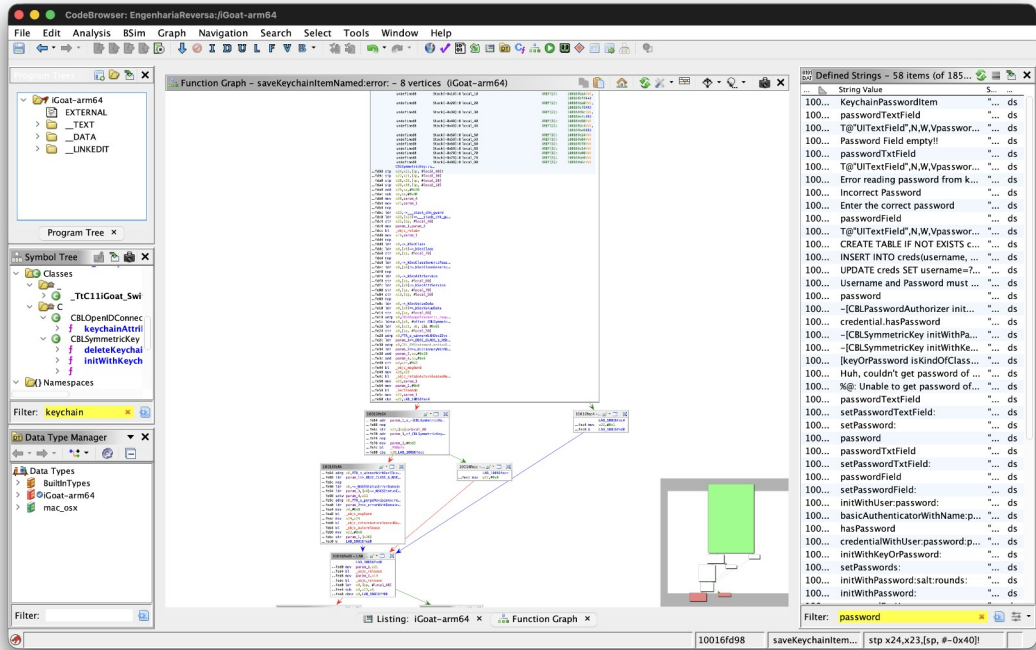
Alvo	Funções recuperadas	Cadeias sensíveis	Símbolos instrumentáveis	Achado dominante
DVIA-v2	19.015	1.288	60	Persistência insegura e validações fracas de fluxo
iGoat-Swift	9.087	298	22	Exposição de parâmetros e trilhas de depuração
UnCrackable L1 e L2	326	44	10	Rotinas de resistência a adulteração e checagens condicionais

Fonte: elaborada pelos autores (2026). Contagens obtidas automaticamente com Radare2 e Rabin2 sobre a fatia ARM64, fixada explicitamente. A linha do UnCrackable consolida os níveis 1 e 2. Dados brutos disponíveis no repositório da pesquisa (FERRONI; SILVA, 2026).

Três dos quatro alvos — iGoat-Swift, UnCrackable L1 e L2 — são binários universais com as fatias armv7 e ARM64, nessa ordem; só o DVIA-v2 tem fatia única ARM64. Sem indicação de arquitetura, o Radare2 e o Rabin2 6.1.6, no host macOS 26.5 ARM64, leram a fatia ARM64; o Ghidra 11.4.2, no mesmo arquivo, carregou a primeira, armv7. A leitura difere: no iGoat-Swift as funções caem de 9.268 para 9.087, e nos dois UnCrackable somados, de 394 para 326. Por isso a fatia é fixada explicitamente.

A Figura 2 mostra como é a saída dessa etapa. À esquerda aparece o grafo de fluxo de controle de uma rotina recuperada pelo Ghidra, com os desvios condicionais que decidem o caminho do programa. À direita estão as cadeias de caracteres localizadas no mesmo binário. São imagens como essa que revelam os pontos de decisão candidatos à observação na etapa seguinte.

Figura 2. Grafo de fluxo de controle e cadeias sensíveis recuperados no Ghidra



Fonte: elaborada pelos autores (2026), a partir da análise do iGoat-Swift.

Procedimento da análise dinâmica

A etapa dinâmica combinou duas ferramentas. O Frida inseriu, em funções de interesse, pequenos trechos de código que registram cada chamada. O Objection permitiu explorar o aplicativo em funcionamento, listando classes e inspecionando o que ele guarda em disco (FRIDA TEAM, 2026; SENSEPOST, 2026). O ciclo foi sempre o mesmo: iniciar o servidor de observação, injetá-lo no processo, carregar os scripts, registrar os interceptadores, capturar argumentos e retornos e exportar tudo para planilha.

Cabe registrar uma correção em relação a versões anteriores deste trabalho. A observação em iOS não usa a ponte para máquina virtual Java, empregada em Android. O ponto de entrada correto é verificar se o ambiente Objective-C está disponível e então instalar o interceptador sobre a implementação do método desejado. Os Quadros 1 e 2 trazem os trechos efetivamente utilizados. Note-se que ambos apenas leem e registram: nenhum altera o valor devolvido pelo aplicativo.

Quadro 1. Trecho de script Frida para observação da validação de transporte

```
// Ambiente de execucao Objective-C do iOS. Nao se usa a ponte Java/Android;
// o ponto de entrada e ObjC.available + ObjC.classes + Interceptor.attach.
if (ObjC.available) {
  var NSURLSession = ObjC.classes.NSURLConnection;
  var seletor = '- dataTaskWithRequest:completionHandler:';
  Interceptor.attach(NSURLSession[seletor].implementation, {
    onEnter: function (args) {
      var request = new ObjC.Object(args[2]);
      var url = request.URL().absoluteString().toString();
      if (url.indexOf("https") === 0) {
        console.log("[TLS] Requisicao observada: " + url);
      }
    }
  });
}
```

Fonte: elaborado pelos autores (2026).

Quadro 2. Trecho de script Frida para observação de consultas a indicadores de ambiente

```
// Intercepta -[NSFileManager fileExistsAtPath:] para observar, sem alterar,
// as consultas do aplicativo a indicadores de comprometimento do sistema.
if (ObjC.available) {
  var NSFileManager = ObjC.classes.NSFileManager;
  var metodo = '- fileExistsAtPath:';
  Interceptor.attach(NSFileManager[metodo].implementation, {
    onEnter: function (args) {
      this.path = new ObjC.Object(args[2]).toString();
    },
    onLeave: function (retval) {
      var indicadores = ["/Applications/Cydia.app", "/bin/bash", "/etc/apt"];
      if (indicadores.indexOf(this.path) >= 0) {
        console.log("[JB] Indicador consultado: " + this.path);
      }
    }
  });
}
```

Fonte: elaborado pelos autores (2026).

Ambiente de execução: simulador e dispositivo físico

A etapa dinâmica pode ocorrer em dois lugares, e eles não são equivalentes. Essa distinção decide como os resultados devem ser lidos. O Simulador iOS roda o aplicativo como um programa comum do macOS. Ele não reproduz o modelo de confiança do iOS: não exige assinatura de código, não tem o mesmo isolamento, não tem Secure Enclave, e a observação acontece diretamente sobre o processo. No aparelho real, é preciso assinatura válida e modo de desenvolvedor, e a observação depende de privilégio elevado ou de reempacotar o aplicativo. Por isso, defesas que procuram sinais de um sistema modificado e operações que dependem do Secure Enclave só fazem sentido no aparelho, como resume a Tabela 2.

Tabela 2. Representatividade das técnicas por ambiente de execução

Cenário	Simulador iOS	Dispositivo físico
Validação de transporte	Observável; sem fixação de certificado em hardware	Observável; representa a cadeia de validação real do aplicativo
Deteção de comprometimento do sistema	Pouco representativo; artefatos ausentes no hospedeiro	Representativo; verificações de artefatos e privilégios efetivamente exercitadas
Keychain	Implementação substituta sobre o hospedeiro; classes de proteção não fiéis	Fiel; classes de acessibilidade e proteção reais
Autenticação local	Limitado; biometria emulada, sem Secure Enclave	Fiel; fluxo de contexto de autenticação e Secure Enclave reais

Fonte: elaborada pelos autores (2026), com base em Apple Inc. (2026) e OWASP Foundation (2024a). A coluna indica a representatividade da observação, e não o resultado experimental.

Os ensaios com números aqui apresentados limitam-se, portanto, ao Simulador e às três técnicas que nele são representativas: validação de transporte, rotinas de Keychain e chamadas de criptografia. Medir a eficácia da verificação de ambiente exigiria um aparelho instrumentado, o que não foi concluído pelas razões técnicas descritas na subseção de limitações. Esse mecanismo aparece no artigo apenas em termos qualitativos, a partir do que a leitura do binário revelou.

Tratamento estatístico

Os ensaios foram organizados em três lotes de trinta tentativas por técnica, somando noventa por cenário. A repetição em lotes permite medir não só a média, mas o quanto os resultados variam entre uma rodada e outra. Como cada tentativa termina em sucesso ou falha, a taxa é tratada como uma proporção. Além da média, o artigo reporta o desvio padrão entre os lotes, o intervalo de confiança de 95% pelo método de Wilson e o coeficiente de variação do tempo de estabilização. Wilson foi preferido à aproximação normal porque se comporta melhor quando a proporção fica próxima de 100% e a amostra é moderada (SCARFONE et al., 2008). Vale fixar o significado da medida: sucesso quer dizer que o interceptador foi instalado sobre a função real do sistema. Não quer dizer que alguma defesa tenha sido contornada.

Validação do instrumento por controles negativos

Uma medida que nunca registra falha levanta uma dúvida legítima: o instrumento é capaz de detectar falha? Sem responder a isso, uma taxa de 100% pode significar tanto que o objeto medido é permissivo quanto que o medidor é cego. Para separar as duas hipóteses, o harness foi submetido a quatro condições em que a falha era esperada, com dez tentativas cada. A Tabela 3 traz o resultado.

Tabela 3. Controles negativos aplicados ao harness de coleta

Controle	Condição imposta	Sucesso	Motivo registrado
A	Interceptor sobre seletor inexistente	0 de 10	hook não instalado (tempo esgotado)
B	Aplicativo encerrado antes da tentativa	0 de 10	processo não encontrado
C	Tempo limite de 0,001 s	10 de 10	nenhum: o controle não se sustentou
D	Nome de processo inexistente	0 de 10	processo não encontrado

Fonte: elaborada pelos autores (2026). Dez tentativas por controle, com a mesma função de medição usada na coleta de produção.

Os controles A, B e D registraram falha em todas as tentativas, cada um com o motivo correspondente gravado na planilha. Isso demonstra que o harness distingue sucesso de falha e rotula a causa, de modo que a taxa da Tabela 4 reflete o objeto medido e não uma limitação do medidor.

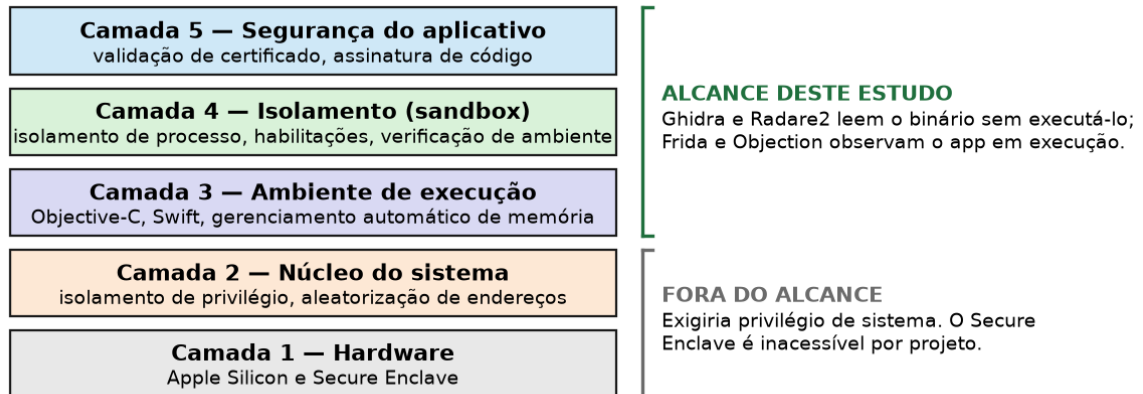
O controle C não se sustentou, e o motivo é informativo. O tempo limite de 0,001 segundo deveria impedir qualquer instalação, mas as dez tentativas foram bem-sucedidas, com tempos entre 0,098 e 0,150 segundo. A inspeção do código explica: o carregamento do script é síncrono e o aviso de interceptor pronto chega durante essa chamada, antes de o laço de espera começar. O tempo limite governa apenas a espera por eventos posteriores, não o caminho de instalação. O controle, portanto, invalida a si mesmo como forma de provocar falha, e não invalida o instrumento. Optou-se por relatá-lo assim, em vez de ajustar o harness até produzir o resultado esperado, porque um controle adaptado para confirmar a hipótese deixa de ser controle. Como efeito colateral, os tempos medidos em C constituem uma verificação independente: são da mesma ordem da média obtida na coleta de produção.

Delimitação de escopo e aspectos éticos

A Figura 3 separa em camadas as defesas da plataforma e mostra até onde cada ferramenta chega. A delimitação mais importante está na camada inferior. Este artigo não discute os limites da engenharia reversa diante do Secure Enclave, porque não há como observar seu firmware, suas operações internas ou as chaves que ele guarda. O Secure Enclave funciona como um processador à parte, com inicialização e memória próprias. As ferramentas empregadas aqui atuam apenas no espaço do usuário e no ambiente do

aplicativo, nunca dentro dele. A fronteira é assumida de propósito, para manter o estudo no que é tecnicamente e legalmente possível.

Figura 3. Arquitetura em camadas do ecossistema de segurança da plataforma



Fonte: elaborada pelos autores (2026), adaptada de Apple Inc. (2026).

A pesquisa não envolveu seres humanos nem animais, o que dispensa submissão a comitê de ética. As questões éticas relevantes são de outra ordem e foram tratadas em três frentes. A primeira é a triagem jurídica dos alvos: só entraram aplicativos abertos, cuja licença autoriza inspeção. A segunda é a finalidade declarada: os scripts observam e registram chamadas, e os resultados não são apresentados como receita para contornar defesas de terceiros. A terceira é o cuidado com dados: nenhum dado pessoal foi coletado e os registros passaram por revisão antes da publicação, para mascarar eventuais identificadores, conforme a Lei Geral de Proteção de Dados (BRASIL, 2018).

RESULTADOS E DISCUSSÃO

Aplicar as duas etapas sobre binários de complexidade diferente produziu uma visão mais completa do que qualquer ferramenta isolada ofereceria. Aplicativos escritos em Swift moderno, com pouco embaralhamento de código, permitiram boa recuperação da estrutura já na leitura do binário. Nos binários com mais verificações internas, foi a observação em funcionamento que sustentou as conclusões.

Desempenho da instrumentação dinâmica

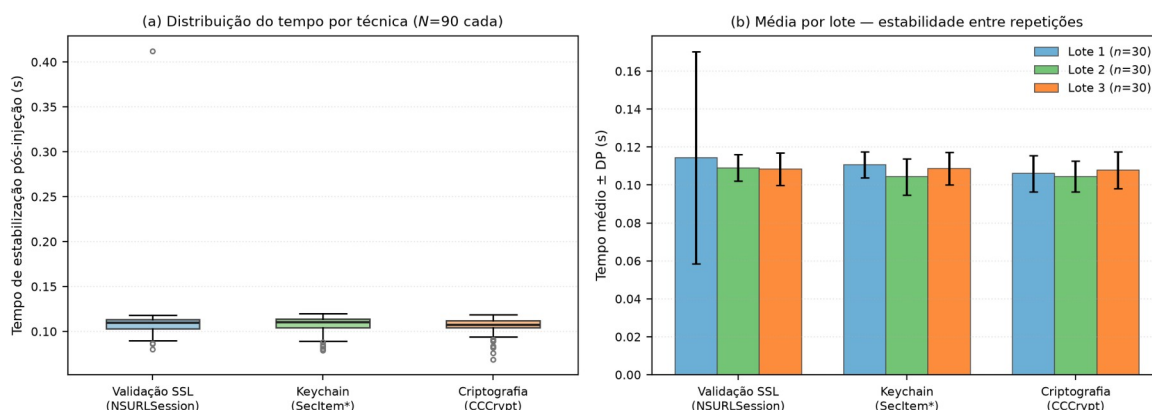
A Tabela 4 traz o desempenho nos três cenários medidos. As três técnicas alcançaram 100,0% de sucesso em noventa tentativas cada, com intervalo de confiança de 95% entre 95,9% e 100,0%. O limite inferior abaixo de 100% não é um erro: mesmo sem nenhuma falha na amostra, o método de Wilson preserva a incerteza compatível com o tamanho da amostra. É preferível a afirmar certeza absoluta a partir de noventa observações.

Tabela 4. Desempenho da instrumentação dinâmica por cenário no Simulador iOS

Cenário	Sucesso (%)	DP (p.p.)	IC 95% (Wilson)	Tempo médio $\pm$ DP (s)	CV (%)
Validação de transporte (NSURLSession)	100,0	0,0	[95,9; 100,0]	0,111 $\pm$ 0,033	29,8
Rotinas de Keychain (SecItem*)	100,0	0,0	[95,9; 100,0]	0,108 $\pm$ 0,009	8,2
Interfaces criptográficas (CCCrypt)	100,0	0,0	[95,9; 100,0]	0,106 $\pm$ 0,009	8,7

Fonte: elaborada pelos autores (2026). Ensaios executados sobre o iGoat-Swift no Simulador iOS. N = 90 por cenário, em três lotes de 30 tentativas. Sucesso corresponde à instalação bem-sucedida do interceptador sobre o símbolo real da interface, e não à evasão de defesas. DP: desvio padrão entre lotes; CV: coeficiente de variação do tempo de estabilização.

Figura 4. Tempo de estabilização após a injeção, por técnica e por lote



Fonte: elaborada pelos autores (2026), a partir dos registros por tentativa (N = 90 por técnica).

A Figura 4 detalha como esses tempos se distribuem. O dado mais informativo não é a média, quase idêntica nas três técnicas e sempre abaixo de 0,12 segundo. É a variação. Keychain e criptografia ficaram em torno de 8% de coeficiente de variação, sinal de comportamento estável entre repetições. A validação de transporte chegou a 29,8%, e essa diferença vem de um único valor extremo, de 0,412 segundo, registrado na primeira tentativa do primeiro lote. O padrão é típico do custo de carregar as bibliotecas de rede pela primeira vez. Depois que o processo aquece, as tentativas seguintes voltam à mesma faixa das demais. Trata-se, portanto, de um efeito do instrumento e do ciclo de vida do programa, não de instabilidade da defesa observada.

Criticidade por família de interfaces

A Tabela 5 agrupa por família as chamadas identificadas na leitura do binário e atribui a cada grupo uma classificação de impacto e de facilidade de exploração. A soma das chamadas corresponde ao total apurado na Tabela 1, o que mantém as duas etapas coerentes.

Tabela 5. Criticidade por família de interface instrumentável

Família de interface	Símbolos	Impacto	Explorabilidade	Nível final
Keychain e segredos (SecItem, SecKey)	18	Alto	Média	Alto

Família de interface	Símbolos	Impacto	Explorabilidade	Nível final
Verificação de ambiente (fileExistsAtPath)	1	Alto	Alta	Crítico
Transporte e certificado (SecTrustEvaluate)	6	Alto	Média	Alto
Criptografia (CCCrypt)	32	Médio	Média	Médio-alto
Rede (NSURLSession)	35	Médio	Alta	Médio-alto

Fonte: elaborada pelos autores (2026). Contagem obtida com Rabin2; a soma é 92, coerente com a Tabela 1. Impacto, explorabilidade e nível final constituem classificação qualitativa adaptada de OWASP Foundation (2024b) e MITRE Corporation (2025).

Persistência de credenciais e gestão de Keychain

No DVIA-v2, a leitura do binário revelou uso extensivo do repositório de preferências para guardar dados de sessão, sem criptografia, com as cadeias de caracteres correspondentes presentes no executável. O alvo não foi acompanhado em funcionamento, de modo que o achado se apoia apenas na leitura estática. Em um aparelho íntegro, outros aplicativos não conseguem ler esses arquivos. O achado ganha peso, porém, se o aparelho estiver comprometido, se a cópia de segurança for insegura ou se a telemetria estiver mal configurada.

O iGoat-Swift trata o assunto de outra forma. Enquanto o DVIA-v2 recorre ao repositório de preferências, o iGoat-Swift usa as interfaces de Keychain: a leitura do binário encontra as quatro operações da família, de inserção a remoção, e nenhuma delas aparece no outro alvo com a mesma finalidade. A comparação entre os dois mostra que a diferença não está na disponibilidade da interface de segurança, que é a mesma no sistema, mas na decisão de projeto sobre onde cada dado é guardado. Conferir se os atributos de proteção escolhidos são os mais restritivos exigiria capturar os argumentos dessas chamadas em execução, o que não foi feito neste ciclo e fica registrado como limitação.

Verificação de ambiente e resistência à depuração

Verificação de ambiente e resistência à depuração costumam ser tratadas juntas, mas respondem a ameaças diferentes. A primeira pergunta se o sistema foi modificado, procurando rastros típicos. A segunda pergunta se o próprio programa está sendo examinado naquele instante, e é a principal defesa contra a observação em tempo de execução. No DVIA-v2, a leitura do binário revelou verificações encadeadas: presença de aplicativos suspeitos, caminhos de execução fora do padrão, permissão de escrita em pastas do sistema e uso de interpretadores com privilégio. Isoladamente fracas, essas checagens ganham força quando combinadas. Medir a eficácia delas, porém, exige aparelho real: no Simulador o resultado não seria representativo, como mostra a Tabela 2.

Para a resistência à depuração, foram procurados os quatro recursos clássicos da plataforma. O primeiro é a chamada `ptrace` com o argumento `PT_DENY_ATTACH`, que instrui o sistema a recusar a conexão de um depurador. O segundo é a consulta `sysctl` sobre `KERN_PROC`, que lê o campo `P_TRACED` da estrutura `kinfo_proc` para descobrir se o processo está sendo acompanhado. O terceiro é `task_get_exception_ports`, que detecta quando um agente externo assume as portas de exceção Mach. O quarto é a checagem agregada de alto nível, normalmente reunida em uma função chamada `isDebuggerAttached`. Cada um foi procurado por três caminhos independentes, que respondem a perguntas diferentes: os símbolos importados dizem se a chamada existe e vem de fora do binário, a tabela de símbolos alcança também o que é local, e as cadeias de caracteres registram nomes e constantes soltos. A Tabela 6 reúne o resultado dessa busca.

Tabela 6. Primitivos de resistência à depuração procurados na análise estática

Primitivo	Mecanismo	DVIA-v2	iGoat-Swift
<code>ptrace</code> com <code>PT_DENY_ATTACH</code>	Nega a anexação de depurador ao processo	Não observado	Não observado
<code>sysctl</code> com verificação de <code>P_TRACED</code>	Inspeciona a estrutura de informação do processo para detectar rastreamento	Não observado	Não observado
<code>task_get_exception_ports</code>	Detecta a captura das portas de exceção Mach por agente externo	Apenas em biblioteca de terceiros	Não observado
<code>isDebuggerAttached</code> (heurística de alto nível)	Combinação de indicadores; sem símbolo direto exportado	Não observado	Não observado

Fonte: elaborada pelos autores (2026). Busca feita com Rabin2 6.1.6 por três caminhos independentes — símbolos importados, tabela de símbolos e cadeias de caracteres.

Nenhum dos quatro primitivos é implementado pelos alvos. As ocorrências que existem não resistem à conferência de origem. No DVIA-v2, `ptrace` aparece apenas como cadeia de caracteres solta, sem importação nem símbolo, e a constante `PT_DENY_ATTACH` não está no binário. As duas importações de `sysctl` e as quatro da família de portas de exceção Mach pertencem ao relator de falhas embutido no aplicativo: os símbolos correspondentes carregam o prefixo da biblioteca e os caminhos de compilação apontam para seus arquivos de origem. Servem para descrever o aparelho e para capturar falhas, não para detectar um depurador. No iGoat-Swift não há ocorrência de nenhum dos quatro, em nenhum dos três caminhos de busca. A ausência é, ela própria, um resultado: a suíte educacional foi construída para expor falhas de armazenamento, criptografia e transporte, e não para exercitar resiliência. Isso qualifica a escolha dos alvos discutida na metodologia — eles reproduzem bem as falhas de aplicativos comerciais, mas não as defesas contra observação, o que reforça a necessidade de estender o estudo a binários de produção.

Atestação de integridade em tempo de execução

Os serviços App Attest e DeviceCheck permitem que o aplicativo gere, por meio do Secure Enclave, uma prova assinada sobre sua integridade e a do aparelho, conferida pelo servidor da aplicação (APPLE INC., 2026). Como a chave nunca deixa o Secure Enclave, alterar valores em tempo de execução não a forja. O estudo trata esses mecanismos como fronteira teórica e não os examina.

Mapeamento aos referenciais MASVS e MITRE ATT&CK Mobile

A classificação usada nas subseções anteriores é agora formalizada. Cada achado foi associado a uma categoria do padrão de verificação de segurança móvel da OWASP (OWASP FOUNDATION, 2024b). Isso liga o trabalho a uma referência reconhecida e permite comparar os resultados com os de outros estudos. A Tabela 7 traz essa correspondência.

Tabela 7. Mapeamento dos achados às categorias do OWASP MASVS

Achado	Categoria MASVS	Criticidade
Credenciais e segredos em armazenamento local	MASVS-STORAGE-1	Alto
Uso inseguro de interfaces criptográficas	MASVS-CRYPTO-1	Médio-alto
Fragilidade na validação de transporte	MASVS-NETWORK-1 e 2	Alto
Verificação de ambiente observável em execução	MASVS-RESILIENCE-1	Crítico
Ausência de resistência à depuração nos alvos	MASVS-RESILIENCE-4	Crítico
Exposição de informações de depuração	MASVS-STORAGE-2	Médio-alto

Fonte: elaborada pelos autores (2026), com base em OWASP Foundation (2024b).

A Tabela 8 completa o quadro pelo lado de quem ataca, ligando cada técnica exercitada em laboratório à correspondente da matriz MITRE ATT&CK for Mobile. O mapeamento é indicativo e adota a perspectiva do analista. O estudo reproduz, de forma controlada e com finalidade defensiva, passos que um adversário daria depois de comprometer o aparelho.

Tabela 8. Correspondência entre as técnicas do estudo e o MITRE ATT&CK Mobile

Técnica do estudo	Técnica ATT&CK	ID
Instrumentação da validação de transporte	Adversary-in-the-Middle	T1638
Observação das verificações de ambiente	Virtualization/Sandbox Evasion: System Checks	T1633.001
Interceptação de rotinas de Keychain	Credentials from Password Store: Keychain	T1634.001
Extração de segredos do armazenamento local	Stored Application Data	T1409

Fonte: elaborada pelos autores (2026), com base em MITRE Corporation (2025).

Desempenho comparado das ferramentas

A Tabela 9 compara as ferramentas e confirma que a contribuição de cada uma é complementar. Os critérios são os previstos no projeto: profundidade da leitura ou da observação, utilidade para examinar defesas, visibilidade sobre o programa em funcionamento e estabilidade de operação.

Tabela 9. Comparação das ferramentas empregadas por tarefa de análise

Ferramenta	Desmontagem	Observação de defesas	Ambiente de execução	Observação técnica
Ghidra	Alta: recuperação de funções, referências cruzadas e pseudocódigo com boa fidelidade semântica em ARM64 e Mach-O	Baixa e indireta: serve para localizar pontos de interceptação e entender precondições	Baixa: atua fora do processo em execução	Melhor resultado na compreensão estrutural e na triagem inicial dos alvos
Radare2	Média a alta: cabeçalhos, símbolos, segmentos e verificação de proteções de compilação	Média: localiza rotinas candidatas e confirma deslocamentos	Baixa: não é seu foco principal	Complementa o Ghidra com leitura de baixo nível e confirmação de metadados
Frida	Média: não é seu propósito, mas mapeia funções em execução e parâmetros reais	Alta: interceptadores em JavaScript observam validações e efeitos em tempo real	Alta: chamadas, argumentos, objetos e estados transitórios do processo	Ferramenta central da observação comportamental
Objection	Baixa: depende do ambiente de execução e de uma sessão de Frida já funcional	Média a alta: exploração interativa e comandos prontos de verificação	Alta: enumeração de classes, armazenamento do aplicativo e memória	Reduz a complexidade operacional em cenários repetitivos

Fonte: elaborada pelos autores (2026). Classificação qualitativa segundo os critérios de avaliação definidos no projeto de pesquisa.

O quadro sustenta um ponto central. O valor do método está menos em qualquer ferramenta específica e mais na correlação entre o que a leitura do binário sugere e o que a execução confirma (EGELE et al., 2012; TAM et al., 2017). Nenhuma das quatro, sozinha, cobre as duas etapas com profundidade suficiente.

Boas práticas sistematizadas

A experiência acumulada permitiu reunir cinco diretrizes para conduzir engenharia reversa acadêmica de forma segura e ética. A primeira é isolar o ambiente, mantendo os ensaios sem tráfego externo e em máquina dedicada à atividade científica. A segunda é conferir a licença antes de qualquer análise, verificando se o alvo é aberto e se autoriza inspeção. A terceira é cuidar dos dados, mascarando identificadores nos registros e descartando o que não for necessário, conforme a Lei Geral de Proteção de Dados. A quarta é versionar scripts, planilhas e roteiros, mantendo-os separados dos binários examinados. A quinta é padronizar o nome dos arquivos gerados, o que reduz confusão e viabiliza auditoria posterior. O ciclo de trabalho que decorre dessas diretrizes tem quatro passos: preparar o binário, ler o binário, observar o programa em lotes repetidos e consolidar os achados com sua classificação.

Ameaças à validade e limitações

Três fatores afetam a validade interna. O primeiro é o momento da injeção: se ela ocorre tarde demais em relação ao início do programa, a falha vem do instrumento e não da defesa. O segundo é a incompatibilidade entre assinaturas de função, que pode interromper a observação. O terceiro é a variação natural do ambiente, incluindo a ordem em que as classes são carregadas. A execução em lotes repetidos e o registro sistemático da dispersão

reduzem esses efeitos, mas não os eliminam. Os controles negativos da Tabela 3 atenuam a principal ameaça desta categoria, ao mostrar que o harness registra falha quando ela ocorre. O que eles não cobrem é a hipótese de falha silenciosa, em que o interceptador se instala sobre um símbolo que não é o pretendido.

Quanto à validade externa, os alvos vêm da suíte educacional da OWASP. São aplicativos vulneráveis de propósito e, por isso, diferentes de programas comerciais em produção, tanto em complexidade quanto em defesas. Os ensaios também ocorreram em versões específicas do sistema operacional, de modo que generalizar exige refazer o ambiente. A limitação mais relevante, porém, é outra: a parte quantitativa restringiu-se ao Simulador. A tentativa de estender a coleta a um aparelho não foi concluída porque o componente de observação interrompia a inicialização do aplicativo na versão utilizada. A alternativa exigiria um aparelho com privilégio elevado, fora do escopo autorizado do projeto. Por isso a eficácia da verificação de ambiente não é reportada em números. Preferiu-se registrar a limitação a estimar valores que não foram observados.

Quanto à validade de construto, a medida principal é o sucesso da observação, isto é, a capacidade de acompanhar uma rotina. Isso não equivale a explorar uma falha em cenário real. Uma interceptação bem-sucedida mostra que o comportamento é observável e potencialmente manipulável, não que um ataque completo tenha sido realizado. A contribuição do estudo está, portanto, menos em generalizar estatisticamente e mais em demonstrar um método: um fluxo de trabalho repetível, delimitado eticamente e documentado. Registre-se ainda que o Frida Stalker, que permitiria remontar o fluxo em rotinas embaralhadas acompanhando a execução instrução a instrução, não foi exercitado neste ciclo.

CONSIDERAÇÕES FINAIS

O objetivo geral foi atingido. As técnicas de engenharia reversa foram aplicadas a aplicativos de código aberto em ambiente controlado, e o comportamento das ferramentas foi avaliado de forma sistemática. Entre os objetivos específicos, quatro foram cumpridos integralmente: a seleção das ferramentas, a montagem do laboratório, a aplicação das duas etapas de análise sobre quatro binários da suíte OWASP e a sistematização das boas práticas. O quinto foi cumprido em parte. A medição consolidou-se para as três técnicas representativas no Simulador, mas não para a verificação de ambiente, que depende de aparelho real.

Dois desvios em relação ao escopo inicial merecem registro explícito. O primeiro foi não analisar binários compilados pelo próprio pesquisador. Eles se mostraram simples e pouco embaralhados demais para exercitar as defesas que interessavam ao estudo, e mantê-

los traria viés sem ganho analítico. O segundo foi restringir a parte quantitativa ao Simulador, já discutido. Nos dois casos a decisão foi tomada durante a execução, com base em evidência, e está documentada. Entende-se que relatar o caminho efetivamente percorrido vale mais, cientificamente, do que preservar a aparência de aderência ao plano inicial.

Os resultados sustentam que combinar leitura de binário e observação em funcionamento é uma estratégia sólida para estudar segurança móvel e apoiar resposta a incidentes. Ghidra e Radare2 reconstruíram bem a estrutura interna dos binários ARM64. Frida e Objection deram visibilidade sobre o comportamento em execução, com desempenho estável em noventa tentativas por técnica e tempo de estabilização abaixo de 0,12 segundo. A hipótese central se confirma: ferramentas de código aberto, em ambiente isolado e com escopo legal definido, bastam para investigação acadêmica no ecossistema iOS. A restrição a alvos abertos, somada à documentação rigorosa, assegurou conformidade ética e reduziu risco jurídico.

Há uma contribuição metodológica que merece destaque. Ao separar de forma explícita o que é observável no Simulador do que só faz sentido em aparelho real, o estudo evita uma confusão comum: tomar a facilidade de observar como sinal de fragilidade da defesa. Reportar a taxa de sucesso como medida de observabilidade, e não de evasão, é o que permite comparar resultados desse tipo entre estudos diferentes.

Quatro caminhos ficam abertos. O primeiro é estender a coleta a um aparelho com privilégio autorizado, o que completaria a avaliação das verificações de ambiente. O segundo é ampliar o conjunto de binários, incluindo aplicações de produção mediante autorização expressa. O terceiro é automatizar parte da triagem inicial. O quarto é aprofundar o estudo das defesas emergentes, com destaque para a atestação apoiada por hardware, que desloca a conferência do aparelho para o servidor e redefine os limites da observação local.

REFERÊNCIAS

APPLE INC. **Licensed application end user license agreement**. Cupertino: Apple Inc., 2023. Disponível em: <https://www.apple.com/legal/internet-services/itunes/dev/stdeula/>. Acesso em: 11 abr. 2026.

APPLE INC. **Apple platform security**. Cupertino: Apple Inc., 2026. Disponível em: <https://support.apple.com/guide/security/welcome/web>. Acesso em: 10 mar. 2026.

BRASIL. **Lei nº 9.609, de 19 de fevereiro de 1998**. Dispõe sobre a proteção da propriedade intelectual de programa de computador, sua comercialização no País, e dá outras providências. Brasília, DF: Diário Oficial da União, 1998. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/19609.htm. Acesso em: 11 abr. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais. Brasília, DF: Diário Oficial da União, 2018. Disponível em:

https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 11 abr. 2026.

CHEN, B. et al. **CREBench**: evaluating large language models in cryptographic binary reverse engineering. 2026. Disponível em: <https://arxiv.org/abs/2604.03750>. Acesso em: 11 abr. 2026.

EAGLE, C.; NANCE, K. **The Ghidra book**: the definitive guide. San Francisco: No Starch Press, 2020.

EGELE, M. et al. A survey on automated dynamic malware-analysis techniques and tools. **ACM Computing Surveys**, New York, v. 44, n. 2, p. 6:1-6:42, 2012.

ENCK, W. et al. A study of Android application security. In: USENIX SECURITY SYMPOSIUM, 20., 2011, San Francisco. **Proceedings** [...]. Berkeley: USENIX Association, 2011. Disponível em: <https://www.usenix.org/conference/usenixsecurity11/study-android-application-security>. Acesso em: 14 ago. 2026.

FERRONI, E.; SILVA, R. C. **Engenharia reversa com aplicativos de código aberto em ambientes experimentais controlados**: dados de pesquisa, scripts de coleta e verificador de números. Versão 1.0.0. 2026. Disponível em: <https://github.com/EnzoFerroni/ios-reverse-engineering-lab>. Acesso em: 20 ago. 2026.

FRIDA TEAM. **Frida**: dynamic instrumentation toolkit for developers, reverse-engineers and security researchers. 2026. Disponível em: <https://frida.re/>. Acesso em: 11 abr. 2026.

GIANCHANDANI, P. **DVIA-v2**: Damn Vulnerable iOS App (Swift). 2025. Disponível em: <https://github.com/prateek147/DVIA-v2>. Acesso em: 20 out. 2025.

HU, L. et al. **Can LLMs deobfuscate binary code?**: a systematic analysis of large language models into pseudocode deobfuscation. 2026. Disponível em: <https://arxiv.org/abs/2604.08083>. Acesso em: 13 jun. 2026.

MITRE CORPORATION. **MITRE ATT&CK for Mobile**. McLean: MITRE, 2025. Disponível em: <https://attack.mitre.org/matrices/mobile/>. Acesso em: 10 mar. 2026.

NATIONAL SECURITY AGENCY. **Ghidra software reverse engineering framework**. Fort Meade: NSA, 2025. Disponível em: <https://github.com/NationalSecurityAgency/ghidra>. Acesso em: 14 ago. 2026.

OWASP FOUNDATION. **OWASP Mobile Application Security Testing Guide (MASTG)**. 2024a. Disponível em: <https://mas.owasp.org/MASTG/>. Acesso em: 15 set. 2025.

OWASP FOUNDATION. **OWASP Mobile Application Security Verification Standard (MASVS)**. 2024b. Disponível em: <https://mas.owasp.org/MASVS/>. Acesso em: 10 mar. 2026.

OWASP FOUNDATION. **MAS crackmes**: UnCrackable apps. 2024c. Disponível em: <https://mas.owasp.org/crackmes/>. Acesso em: 14 ago. 2026.

OWASP FOUNDATION. **iGoat-Swift**: a damn vulnerable Swift application for iOS. 2025. Disponível em: <https://github.com/OWASP/iGoat-Swift>. Acesso em: 20 out. 2025.

PORDANESH, S.; TAN, B. **Exploring the efficacy of large language models (GPT-4) in binary reverse engineering**. 2024. Disponível em: <https://arxiv.org/abs/2406.06637>. Acesso em: 11 abr. 2026.

PROMON SECURITY RESEARCH TEAM. **App threat report 2026 Q1**: the state of code obfuscation against AI. Oslo: Promon, 2026. Disponível em: <https://promon.io/security-news/app-threat-report-2026-q1-the-state-of-code-obfuscation-against-ai>. Acesso em: 14 ago. 2026.

RADARE2 PROJECT. **Radare2**: Unix-like reverse engineering framework and command-line toolset. 2024. Disponível em: <https://rada.re/n/>. Acesso em: 20 out. 2025.

SCARFONE, K. et al. **Technical guide to information security testing and assessment**: NIST Special Publication 800-115. Gaithersburg: National Institute of Standards and Technology, 2008. Disponível em: <https://csrc.nist.gov/pubs/sp/800/115/final>. Acesso em: 14 ago. 2026.

SCHIRRU, L. A engenharia reversa de software no Brasil: uma análise sobre a sua viabilidade legal. **Cadernos de Prospecção**, Salvador, v. 7, n. 3, p. 345-355, 2014.

SENSEPOST. **Objection**: runtime mobile exploration. 2026. Disponível em: <https://github.com/sensepost/objection>. Acesso em: 11 abr. 2026.

SIKORSKI, M.; HONIG, A. **Practical malware analysis**: the hands-on guide to dissecting malicious software. San Francisco: No Starch Press, 2012.

SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

TAM, K. et al. The evolution of Android malware and Android analysis techniques. **ACM Computing Surveys**, New York, v. 49, n. 4, p. 1-41, 2017.

Contatos: enzo.ferroni@mackenzista.com.br e rodrigoc.silva@mackenzie.br